

Secure Card Payment System

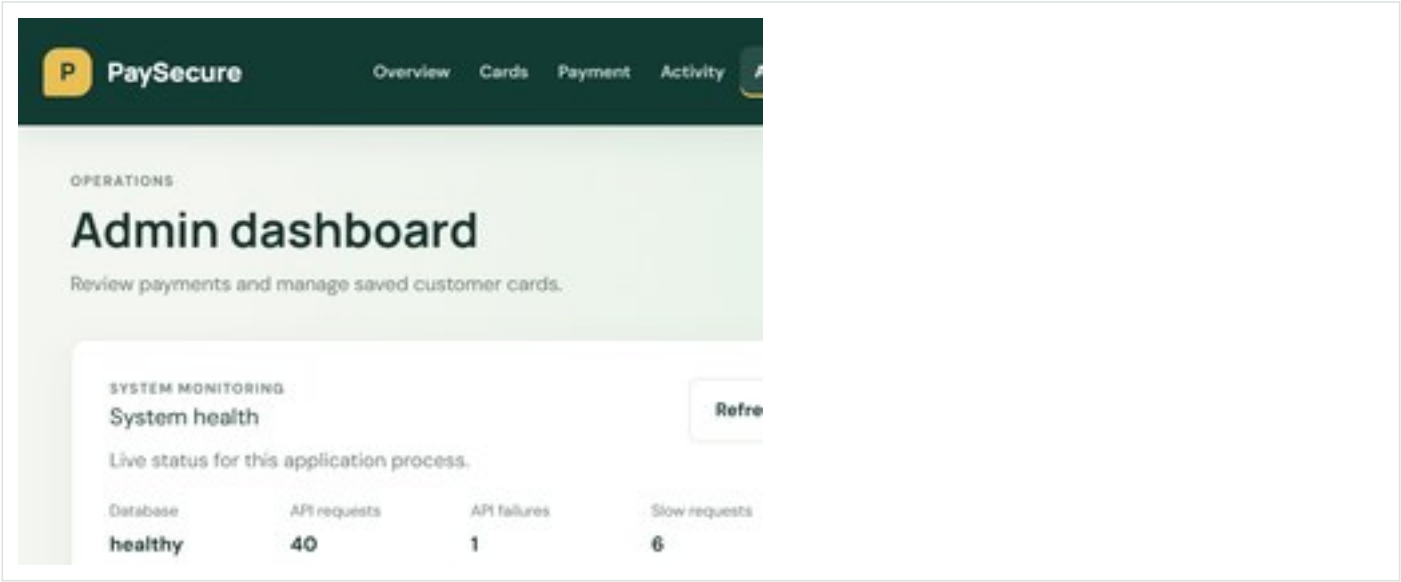
RBAC, fraud detection, transaction analytics, monitoring, and operational reporting

A Django + FastAPI + React system for simulated card payments, with role-aware operations, reviewable fraud alerts, and auditable admin actions.

Executive summary

The delivered application combines customer payment and card workflows with staff operations. Django owns identity, card and transaction data, permissions, fraud review, and audit records; FastAPI provides the payment-facing service; React presents customer and admin dashboards.

SECURITY	FRAUD	VISIBILITY	OPERATIONS
Three staff roles with scoped permissions	Rule checks, alerts, review states, and email	Analytics, search, charts, and exports	Health metrics, structured audit, Docker



Admin dashboard evidence: service health, export actions, payment metrics, and card operations.

Scope note: payment processing is simulated; the application does not connect to a real payment gateway.

01 | Security, roles & audit

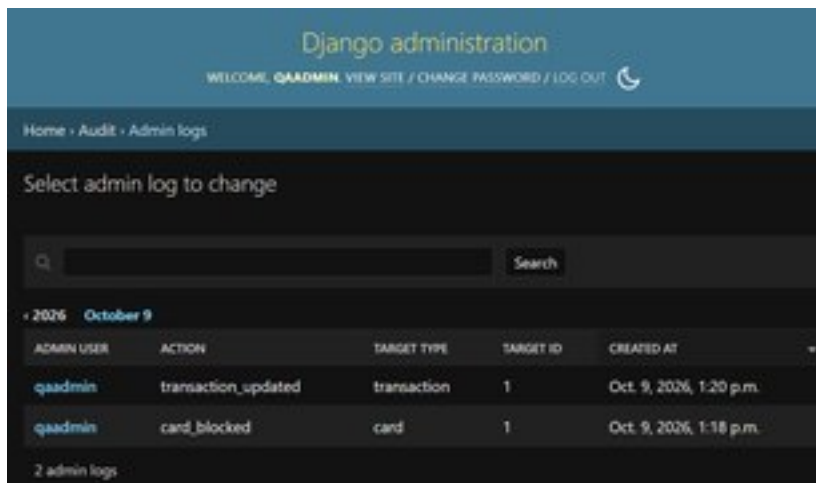
Django group permissions enforce who may view or change cards, transactions, analytics, and fraud reviews. Customer APIs remain authenticated and scoped to the signed-in user's own records.

Role	Allowed operations	Explicit restrictions
Admin	View all cards/transactions; block, unblock, remove cards; change credit limits; analytics and exports; review fraud alerts.	Sensitive actions are recorded; no full PAN or CVV is stored.
Support	View cards/transactions; block or unblock cards; analytics; view and review fraud alerts.	Cannot change credit limits, remove cards, or initiate customer payments.
Read-Only	View operations data, analytics, and fraud alerts.	No card changes, payment initiation, or fraud-alert review.

Audit design

The **admin_logs** table stores actor, action, target, timestamp, details, and structured before/after changes. Card block/unblock, credit-limit changes, transaction edits, and fraud-alert reviews create records. Admin users may also add immutable **manual_note** entries; these are clearly distinguished from system-generated events.

Privacy by design: card records retain a masked number and last four digits; audit details avoid card PAN and CVV.



Django Admin audit list showing card and transaction actions with actor and target.

02 | Fraud detection & alerting

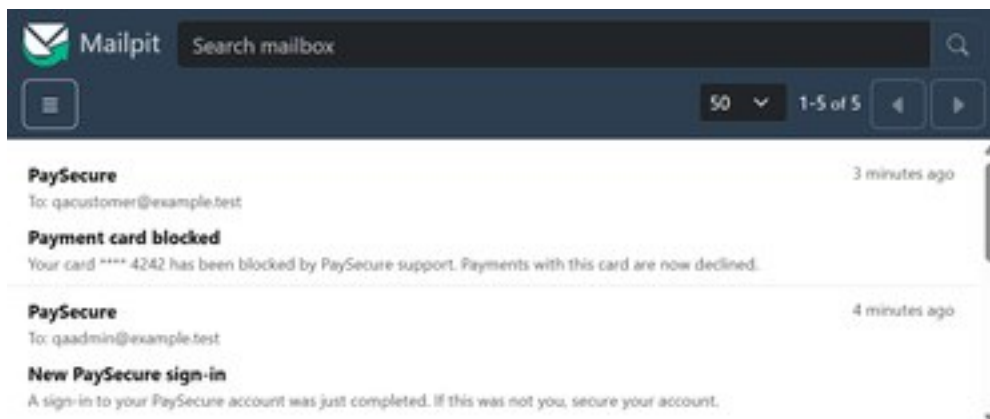
The fraud evaluator runs as part of transaction creation. It checks a rolling 10-minute activity window and creates a linked review record when a rule is matched.

Rule	Detection condition	System response
Repeated high value	Three or more transactions of at least ■5,000 in ten minutes.	Flag transaction; create alert with rule code.
Rapid location change	Source IP fingerprint differs from a recent attempt in the window.	Flag transaction; alert for review.
Rapid device change	Device fingerprint differs from a recent attempt in the window.	Flag transaction; alert for review.

Detection and response flow

Payment attempt	Evaluate	Record	Notify & review
User, card, amount, device	Recent attempts + rule window	Fraud status + alert	Email + staff decision

- Source IP and device identifiers are HMAC-fingerprinted before persistence.
- Alerts retain rule codes, status, reviewer, detection time, and review time.
- Email is queued after transaction commit for the account holder and active operations recipients.
- A flagged payment is reviewable; detection does not automatically decline the simulated payment.



Mailpit evidence from the end-to-end run, including payment and card-operation notifications.

Location is inferred from source IP changes, not precise geolocation. Fingerprint comparison is a risk signal, not proof of fraud.

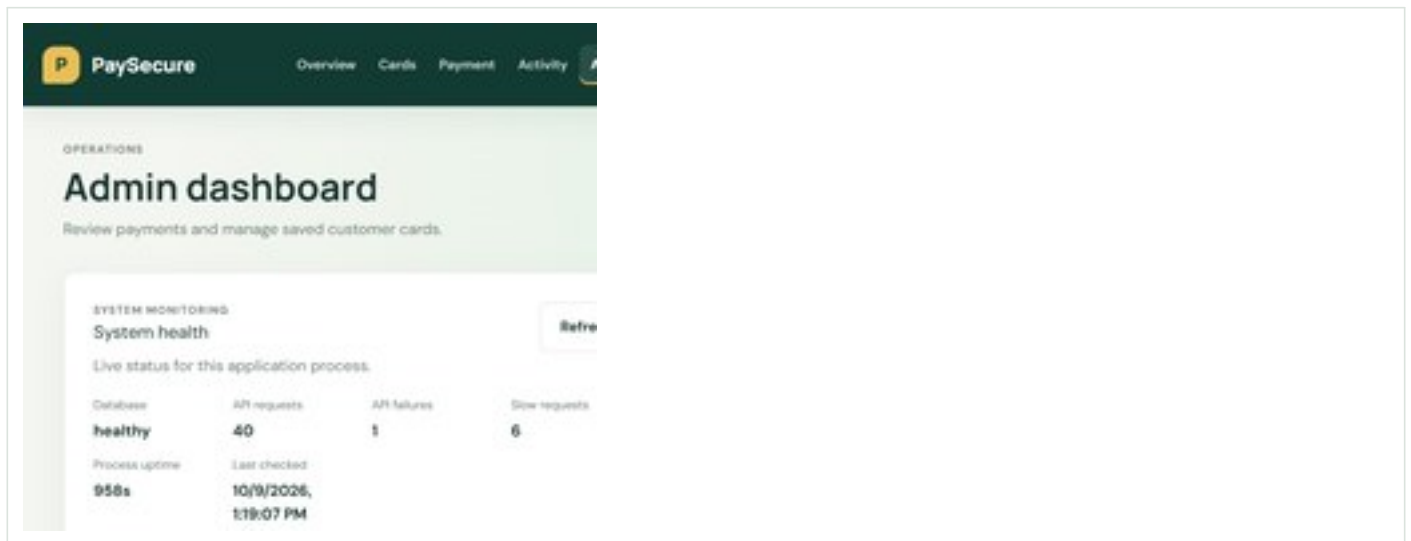
03 | Analytics & transaction search

The analytics API summarizes successful spending and transaction activity for customer and staff dashboards. The frontend presents spending trends and category/status breakdowns, plus card utilization.

Analytics capability	Definition / behavior
Monthly spending	Six months of successful transaction totals, grouped by month.
Category breakdown	Expense totals across categories such as travel, shopping, bills, and dining.
Credit utilization	Successful credit-card spending compared with the configured credit limit.
Activity context	Status totals and recent daily payment attempts support dashboard interpretation.

Search, pagination & query efficiency

- Filter by date range, minimum/maximum amount, status, and masked card number or last four digits.
- Sort by created date, amount, status, reference, or category; prefix with “-” for descending order.
- Server pagination returns count/next/previous/results; default 20, maximum 100 rows per page.
- Composite user/date/status and fraud/date indexes support common filters and review workflows.



Dashboard evidence for live monitoring and reporting; the dashboard also provides CSV/PDF exports.

Data semantics: monthly analytics use successful payments; total-spend fields may include other statuses where the API definition says so.

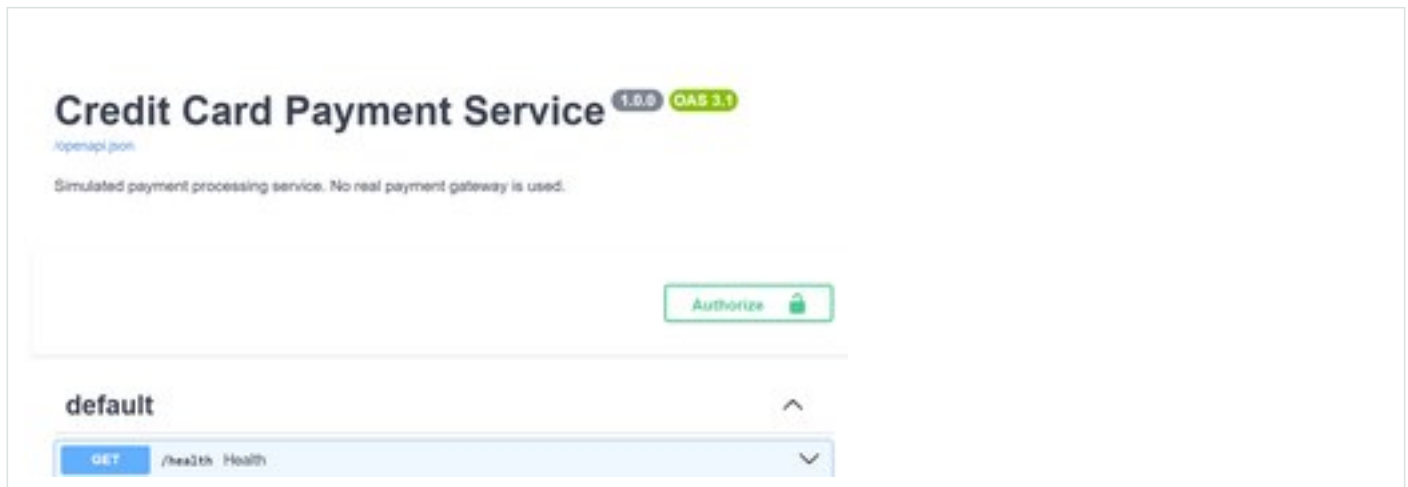
04 | Monitoring, exports & APIs

Request middleware in Django and FastAPI records method, path, status, and duration. Slow requests are flagged using a configurable threshold; failures and exceptions are logged for diagnosis.

Operations surface	What it provides
System health	Database connectivity, process-local request/failure/latency counters, uptime, and slow-request totals.
Analytics exports	Role-authorized CSV and PDF summaries from the admin reporting endpoint.
Django REST Swagger	Browsable API endpoints and generated OpenAPI schema for the Django service.
FastAPI Swagger	Browsable payment-service API, schemas, and health/version endpoints.
Mailpit	Local SMTP capture for payment and operational notification verification.

Validated service surfaces

Surface	Observed result in live Docker verification
Frontend	HTTP 200; sign-in, checkout, dashboard flows exercised.
Django API docs / schema	HTTP 200; endpoints and generated schema available.
FastAPI docs / OpenAPI / health	HTTP 200; payment and health surfaces available.
Django Admin	Sign-in and audit-history pages exercised.
Mailpit	HTTP 200; end-to-end messages received.



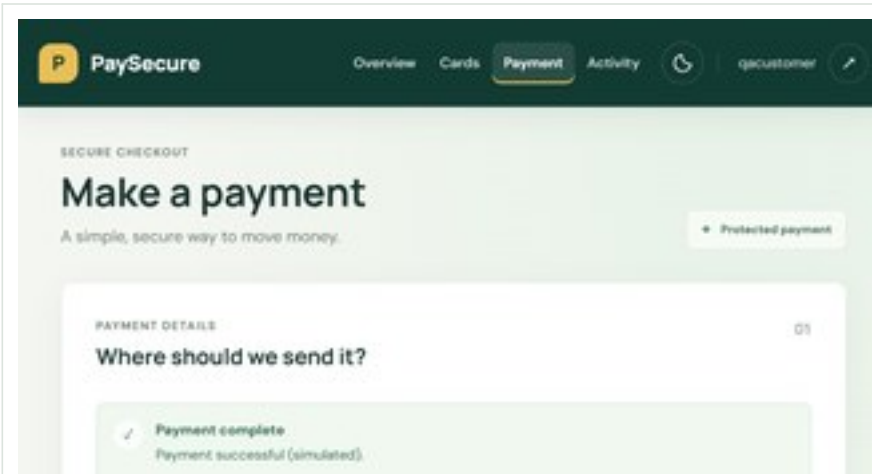
FastAPI Swagger UI with health, dashboard, and simulated payment operations.

Monitoring counters are process-local and reset on restart; production multi-worker deployments should forward metrics and logs to centralized observability.

05 | Validation & delivery

Implementation is documented in the project README and Django migrations. The service stack runs with Docker Compose and retains MySQL data in a named volume.

Verification	Result
Django automated tests	46 tests passed in the latest run after Admin add-flow changes.
FastAPI automated tests	15 tests passed in the prior integration run.
Frontend automated tests	11 tests passed in the prior integration run.
Docker end-to-end	Compose services started; payment, card action, audit, email, docs, and health flows exercised.
Whitespace / patch check	git diff --check passed.



End-to-end checkout evidence. Card/payment values shown are synthetic and the charge is simulated.

Run locally: docker compose up --build. MySQL is published on host port 3307 by default to avoid a common Windows port-3306 conflict; containers use MySQL port 3306 internally. Create a Django superuser before signing in to Admin.

Limitations: no real payment gateway is connected; anomaly rules are deterministic starter rules; health counters are process-local; email delivery depends on configured SMTP (Mailpit in the demo).